Structured programming is a procedural design philosophy that constrains the number and type of logical constructs used to represent algorithmic detail. The intent of structured programming is to assist the designer in defining algorithms that are less complex and therefore easier to read, test, and maintain.

Component-based software engineering identifies, constructs, catalogs, and disseminates a set of software components for an application domain. These components are then qualified, adapted, and integrated for use in a new system. Reusable components should be designed within an environment that establishes standard data structures, interface protocols, and program architectures for each application domain.

PROBLEMS AND POINTS TO PONDER

11.1. The term *component* is sometimes a difficult one to define. First provide a generic definition, and then provide more explicit definitions for object-oriented and traditional software. Finally, pick three programming languages with which you are familiar and illustrate how each defines a component.

11.2. Why are control components necessary in traditional software and generally not required in object-oriented software?

11.3. Describe the OCP in your own words. Why is it important to create abstractions that serve as an interface between components?

11.4. Describe the DIP in your own words. What might happen if a designer depends too heavily on concretions?

11.5. Select three components that you have developed recently, and assess the types of cohesion that each exhibits. If you had to define the primary benefit of high cohesion, what would it be?

11.6. Select three components that you have developed recently, and assess the types of coupling that each exhibits. If you had to define the primary benefit of low coupling, what would it be?

11.7. Develop (1) an elaborated design class, (2) interface descriptions, (3) an activity diagram for one of the operations within the class, and (4) a detailed statechart diagram for one of the *SafeHome* classes that we have discussed in earlier chapters.

11.8. What is a WebApp component?

11.9. Select the code from a small software component and represent it using an activity diagram.

11.10. Why is "chunking" important during the component-level design review process?

Design element: Quick Look icon magnifying glass: © Roger Pressman

CHAPTER

12 User Experience Design

We live in a world of high-technology products, and virtually all of them—consumer electronics, industrial equipment, automobiles, corporate systems, military systems, mobile apps, WebApps, video games, and virtual reality simulations—require human interaction. If a product is to be successful, it must provide a positive user experience (UX). The product needs to exhibit good *usability*—a qualitative measure of the ease and efficiency with which a human can employ the functions and features offered by the high-technology product. A product should incorporate *accessibility* considerations such as assistive technologies when its specified users include people with a range of disabilities within a specified context of use.

KEY CONCEPTS

accessibility 237
accessibility guidelines 259
command labeling 260
customer journey map 244
error handling 260
golden rules 234
help facilities 260
information architecture 235
interface analysis 243
interface consistency 240
interface design 250
internationalization 260
memory load 239
response time 259
storyboard 261
task analysis 247
usability guidelines 257
user experience analysis 243
user interaction design 236
user scenario 261
visual design 237



QUICK LOOK

What is it? User experience (UX) design is the process of enhancing user satisfaction with a product by creating a usable, accessible, and pleasurable interaction between product and its users.

Who does it? A software engineer designs the user experience and user interface assisted by knowledgeable stakeholders.

Why is it important? If software is difficult to use, if it forces you into mistakes, or if it frustrates your efforts to accomplish your goals, you won't like it, regardless of the computational power it exhibits, the content it delivers, or the functionality it offers. The user experience has to be right because it molds a user's perception of the software.

What are the steps? User interface design begins with the identification of user, task, and environmental requirements. These form the basis for the creation of a screen layout and navigation pathways through the information architecture.

What is the work product? User persona and scenarios are created based on the desired customer journey. Low-fidelity prototypes and digital interface prototypes are developed, evaluated, and modified in an iterative fashion.

How do I ensure that I've done it right? An interface prototype is "test driven" by the users, and feedback from the test drive is used for the next iterative modification of the prototype.

For the first three decades of the computing era, usability was not a dominant concern among those who built software. In his classic book on design, Donald Norman [Nor88] argued that it was time for a change in attitude: "To make technology that fits human beings, it is necessary to study human beings. But now we tend to study only the technology. As a result, people are required to conform to technology. It is time to reverse this trend, time to make technology that conforms to people."

As technologists studied human interaction, two dominant issues arose. First, a set of *golden rules* (discussed in Section 12.2) were identified. These applied to all human interaction with technology products. Second, a set of *interaction mechanisms* were defined to enable software designers to build systems that properly implemented the golden rules. These interaction mechanisms, collectively called the *user interface,* have eliminated some of the many egregious problems associated with human interfaces. But even today, we all encounter user interfaces that are difficult to learn, difficult to use, confusing, counterintuitive, unforgiving, and in many cases, totally frustrating. Yet, someone spent time and energy building each of these interfaces, and it is not likely that the builder created these problems purposely.

User experience design is a set of incremental process activities that help the development team and the project stakeholders focus on providing a positive experience for users of the software product. UX design is broader than user interface design and usability or accessibility engineering. It must begin early in the project life cycle if it is to be effective. Developers waiting until the end of a project to add user interface functionality are unlikely to provide a pleasurable experience for users.

In this chapter we will focus on user interface design issues in the context of user experience design. Readers wishing a more detailed coverage of UX should examine books by Shneiderman [Shn16], Nielsen [Nei93], and Norman [Nor13].

12.1 User Experience Design Elements

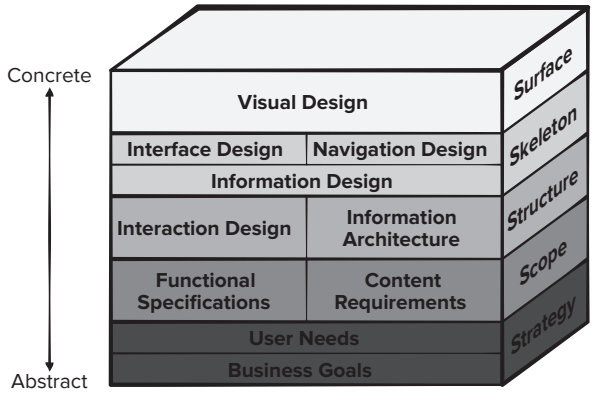
User experience design tries to ensure that no aspect of your software appears in the final release candidate without the explicit decision of the development team and other stakeholders to include it. This means taking into account every reasonable user action and expectation during every step of the development process. To make the task of crafting a positive user experience more manageable, Garret [Gar10] suggests breaking it down into component elements: strategy, scope, structure, skeleton, and surface. The relationships among these component elements and subcomponents are show in Figure 12.1.

The Garrett's organization of UX design can be interpreted as follows for software product development:

- **Strategy.** Identifies user needs and customer business goals that form the basis for all UX design work (Section 12.4)
- **Scope.** Includes both the functional and content (e.g., information, media, services) requirements needed to realize a feature set consistent with the project strategy

FIGURE 12.1

User experience design elements



- **Structure.** Consists of the interaction design [e.g., how the system reacts in response to user actions (Section 12.1.2)] and information architecture [e.g., the organization of the content elements (Section 12.1.1)]
- **Skeleton.** Comprised of three components: information design (e.g., presentation of content in a way to make it understandable to the user), interface design [e.g., arranging interface screen objects to allow the user to work with the system functionality (Section 12.5)], navigation design (e.g., the set of screen elements that allow users to traverse the information architecture)
- **Surface.** Presents visual design or the appearance of the finished project to its users (Section 12.1.4)

Several cross-cutting aspects of UX design are of particular interest to software engineers: information architecture, user interaction design, usability engineering, and visual design.

12.1.1 Information Architecture

As an architectural designer, you must identify information (content) architecture and software architecture. The term *information architecture* is used to connote structures that lead to better organization, labeling, navigation, and searching of content objects. Content architecture focuses on the manner in which content objects (or composite objects such as screens or widgets) are structured for presentation and navigation. Software architecture addresses the manner in which the application is structured to manage user interaction, handle internal processing tasks, effect navigation, and present content.

Architectural design (Chapter 10) is tied to the goals established for a software product, the content to be presented, the users who will visit, and the navigation philosophy that has been established. In most cases, architecture design is conducted in parallel with interface design, aesthetic design, and content design. Because the software architecture may have a strong influence on navigation, the decisions made during this design action will influence work conducted during navigation design. In many cases, a subject matter expert is needed to help project stakeholders organize the content items for efficient assimilation and traversal by product users.

12.1.2 User Interaction Design

Interaction design focuses on the interface between a product and its user. Not too many years ago, the only way for a user to interact with a computer system was typing input on a keyboard and reading output on a display screen of some kind. Today the modes of input and output are quite varied and may include voice input, computer speech generation, touch input, 3D printed output, immersive augmented reality experiences, and sensor tracking of users within their environment. Devices like this are needed to give users ways to control a computer system. Oftentimes these interaction devices interfere with the product providing a natural and pleasurable user experience.

User interaction should be defined by the stakeholders in the user stories (Chapter 7) created to describe how users can accomplish their goals using the software product. This suggests that user interaction design should also include a plan for how information should be presented within such a system and how to enable the user to understand that information. It is important to recall that the purpose of the user interface is to present just enough information to help the users decide what their next action should be to accomplish their goal and how to perform it.

We will describe the user interface design process in more detail in Section 12.5, but initially, there are important questions user interaction designers must ask when devising user interfaces:[1]

- What can users do with a mouse, finger, or stylus to interact with the interface directly?
- What about the appearance (e.g., color, shape, size) gives users clues about how the user interaction functions?
- What information do you provide to let users know what will happen before they perform an action?
- Are there any constraints put in place to help prevent errors?
- Do error messages provide a way for users to correct a problem or explain why an error occurs?
- What feedback do users get once an action is performed?
- Are the interface elements a reasonable size to facilitate interaction?
- What familiar or standard formats should be used to display information and accept input?

12.1.3 Usability Engineering

Usability engineering is part of UX design work that defines the specification, design, and testing of the human-computer interaction portion of a software product. This software engineering action focuses on devising human-computer interfaces that have high usability. Usability engineering provides structured methods for achieving efficiency and elegance in interface design. Terms like *user friendliness* do not provide much guidance here since this is often a very subjective judgment. If developers focus on making a product easy to learn, easy to use, and easy to remember over time, usability can be measured quantitatively and tested for improvements in usability.

1 See https://www.usability.gov/what-and-why/interaction-design.html.

Accessibility is another aspect of usability engineering that should be considered when designing user interactions with the software. *Accessibility* is the degree to which people with special needs (e.g., sight impaired, deaf, elderly, cognitively impaired) are provided with a means to perceive, understand, navigate, and interact with computer products. The goal of accessibility design is to provide hardware or software tools that can remove barriers that may prevent users from successfully completing tasks supported by the software. Usability and accessibility are discussed in greater detail in Section 12.7.

12.1.4 Visual Design

Visual design, also called aesthetic design or graphic design, is an artistic endeavor that complements the technical aspects of the user experience design. Without it, a software product may be functional, but unappealing. With it, a product draws its users into a world that embraces them on a visceral, as well as an intellectual level.

But what is aesthetic? There is an old saying, "Beauty exists in the eye of the beholder." This is particularly appropriate when aesthetic design for many games or mobile apps is considered. To perform effective aesthetic design, you should return to the user hierarchy developed as part of the requirements model (Chapter 8) and ask, "Who are the product's users and what 'look' do they desire?"

Graphic design considers every aspect of the look and feel of a web or mobile app. The graphic design process begins with screen layout and proceeds into a consideration of global color schemes; type fonts, sizes, and styles; the use of supplementary media (e.g., audio, video, animation); and all other aesthetic elements of an application. Not every software engineer has artistic talent. If you fall into this category, hire an experienced graphic designer for aesthetic design work.

SAFEHOME



Graphic Design

The scene: Doug Miller's office after the first *SafeHome* room layout interface prototype review.

The players: Doug Miller, SafeHome software engineering project manager, and Vinod Raman, member of the *SafeHome* software engineering team.

The conversation:

Doug: What's your impression of the new room layout design?

Vinod: I like it, but more importantly, our customers like it.

Doug: How much help did you get from the graphic designer we borrowed from marketing?

Vinod: A great deal actually. Marg has a great eye for page layout and suggested an awesome graphic theme for the app screens. Much better than what we came up with on our own.

Doug: That's good. Any issues?

Vinod: We still have to create an alternate screen to take accessibility issues into account for some of our visually impaired users. But we would've had to do that for any app design we had.

Doug: Can we use Marg for that work as well?

Vinod: Yeah, she has a good understanding of usability and accessibility.

Doug: OK, I'll set it up with marketing and borrow her a little longer.